

Telemetry system with inconsistent metric names

front: five sentences first, then the two-part follow-up · SD Q5 §10 as the Sep 18 onsite asked it

Anthropic question this is: SD Q5 data infrastructure, §10 telemetry variant (prachub onsite Sep 18, 2026: "telemetry reconciliation", inconsistent names for one metric, schema evolution). **Same spine as:** §9 the recruiter's platform blurb (1point3acres, Sep 7 screen) · §11 the analytics dashboard (Tyler #4, Sep 29: one output topic, three consumers). **Anthropic's own probes still to expect:** estimate the infrastructure per part · what fails and what happens · what changes at 10× · serve by sensitivity. **Tyler #6 graded:** the CAP sentence, the scrubbed normaliser, the missing raw writer and alerting.

1 REQUIREMENTS 0–5 min · numbers at minute 5

- **Five sentences, before anything else:** clients batch to an ingestion gateway that validates, stamps an event id, appends to Kafka and acks only after the append · three consumers on the same log: raw writer to cold storage, stream processor that canonicalises and rolls up per series into 1-min buckets, alert evaluator · rollups in a time-series store, raw kept for replay · names resolved at ingest through a versioned alias registry, raw name kept, unknowns quarantined · history fixed by replaying raw, query-time alias meanwhile.
- **Ask: what is an event? who queries (engineers, dashboards, alerting)? freshness? retention raw vs rollups? cardinality (metrics × labels)?**
- **Technical challenges:** 100k clients · 1M events/s · ingest never blocks or rejects · at-least-once, dedupe by event_id · queryable <60 s · raw kept · **availability over consistency.**
- **The CAP sentence:** "Recording an event is a deposit, not a withdrawal: nothing waits on a previous write, so a queue sits in front of the store and every event is accepted now. If you need a queue, you are choosing availability."
- **Numbers:** 1M/s × 200 B = 200 MB/s ≈ 17 TB/day raw · week hot 120 TB · 10M series (100k × 100) → rollups per series · 20–100 Kafka partitions · 60 s freshness budget.

2 CORE ENTITIES 5–9 min · things you can count

- **Event**(event_id, client_id, name_raw, canonical_id, labels, value, ts, schema_version)
- **Series**(canonical_id + labels) · **Rollup**(series, bucket_start, granularity, count / sum / min / max / p99)
- **Alias**(raw_name, canonical_id, valid_from_version, mapped_by) — the registry row
- **AlertRule**(series selector, condition, window, channel) · **Client**(id, agent_version)
- Tyler: not "isolation", not "receipt"; only what the system remembers or counts.

3 API 9–13 min · CRUD each entity; the yeses are the endpoints

- POST /events [{event_id, client_id, name, labels, value, ts}] → 202 after the Kafka append; idempotent on event_id
- GET /metrics?name&labels&from&to&step&agg → buckets from the rollup store; raw below the smallest rollup
- PUT /aliases {raw_name, canonical} · GET /aliases/unmapped (the review queue)
- POST /alerts · GET /alerts/{id}/state
- "There is not much API here; they send generic events of many kinds." Two hot endpoints, the rest is registry and rules.

4 DATA FLOW 13–20 min · numbered on the board

- Client buffers 1–5 s → **1** POST batch → gateway validates, dedupes, looks up the alias (cached), stamps canonical_id + mapping version → **2** append to Kafka → **202 to the client only now.**
- **3a** raw writer → object storage by day and client · **3b** stream processor keys by series, 1-min buckets → **4** time-series store, **5** output topic · **3c** alert evaluator on the buckets → pager.
- **6** dashboards query the time-series store; ad-hoc and replay read the lake. Unknown name → quarantine topic → review queue → human maps → registry version N+1.

5 HIGH-LEVEL DESIGN 20–33 min · the board

- Clients → API gateway → ingestion (= batching; validate, dedupe, alias) → Kafka (append-only, partitioned by client) → raw writer → cold object storage · stream processor / rollup → time-series store (hot 7 d) + output topic · alert evaluator → channels · alias registry + quarantine beside ingest · backfill job from the lake · metrics API and dashboards.
- **Fan-in before Kafka, fan-out after it.** Every consumer reads the same log at its own pace; Kafka absorbs their lag.
- **Batching is a failure window:** ack after the append; the client retries; dedupe makes the retry safe.

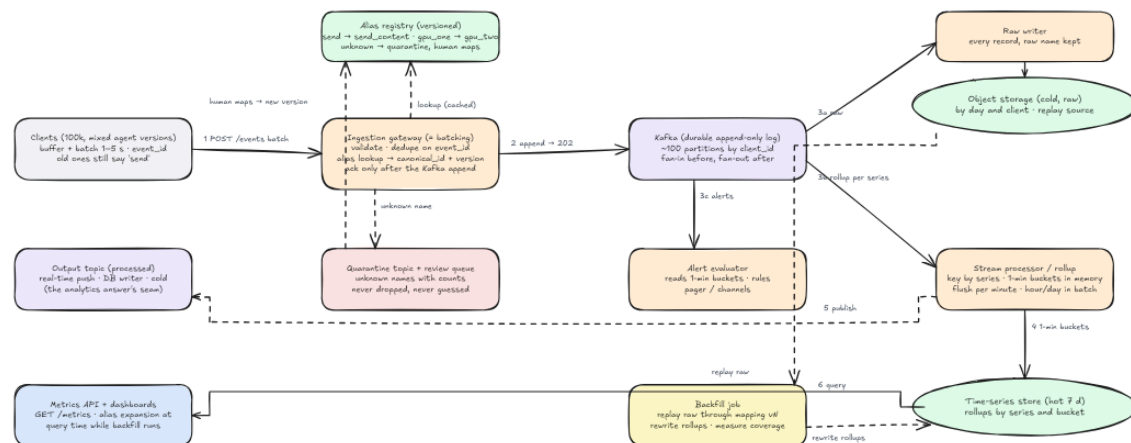
6 DEEP DIVES 33–48 min · the follow-up in two parts, then the probes

- **Stop new data landing under old names:** resolve at ingest via the versioned alias registry (send → send_content, gpu_one → gpu_two); keep the raw name on the record; unknowns to quarantine with counts; never fuzzy-match; deprecate old names by watching agent-version adoption.
- **Fix the history:** replay raw from the lake through mapping vN into canonical partitions, rewrite rollups, measure coverage and collisions; meanwhile the metrics API expands send_content = send ∪ send_content at query time, so one series and no gap.
- **Rollups at 1M/s:** per series with a memory budget, pre-aggregate 10 s in the agent, flush per minute, hour/day in batch; some metrics (errors, XIDs) never rolled up.
- **Backpressure:** Kafka absorbs consumer lag; if ingest is overloaded shed by tier (debug first, never errors) and sample hot series; never block the client.
- **Failure, growth:** idempotent by event_id, dead-letter the rest · bad mapping is versioned, re-run from raw, partition by day and board

The board at minute thirty

back: the event path numbered, the follow-up placed on it

- client, compact, tier to cold · per-metric cardinality limit that pages.
- Bow:** the five sentences, then "canonicalise at ingest and keep



Follow-up, two parts (Tyler's split). 1 Stop new data landing under old names: resolve at ingest through the versioned alias registry, keep the raw name on the record, quarantine unknowns. 2 Fix the history: replay raw from the lake through mapping v1 into canonical partitions, rewrite the rollops, measure coverage and collisions; meanwhile the metrics API expands send_content = send U send_content at query time so the user sees one series and no gap.

Numbers to say 3M/s = 200 B = 200 MB/s = 17 TB/day · 10M series → rollops per series, not per event · 20-100 Kafka partitions · 60 s freshness budget · shed by tier (debug first, never errors) rather than block the client.

Tyler's lines: If you need a queue, you are available over consistent: entities are things you can count; CRUD each one and the gses are the API · latching is a failure window: ask after the append · roll up per event type with a memory budget.

Legend
□ clients □ services □ log, topics □ stores, registry □ quarantine □ batch □ serving
 solid = the event path, numbered dashed = registry, quarantine, replay, backfill